

LANGUAGE REFERENCE MANUAL

LEXICO

A Human-Oriented Programming Language

Complete Language Documentation

From First Program to Advanced Object-Oriented Design

Author: **Haythem Gharbi**

Faculty of Sciences of Monastir

Version 1.0 — Academic Year 2025–2026

Table of Contents

1.	Introduction	3
2.	Installation & Setup	4
3.	Quick Start	5
4.	Language Fundamentals	6
5.	Statements in Depth	8
6.	Expressions and Operators	11
7.	Collections: Meshes, Tables, Matrices	13
8.	Routines and Classes	16
9.	File I/O	18
10.	Modules — <code>load</code>	20
11.	AI-Assisted Error Recovery	21
12.	Diagnostics Reference	26
13.	Complete Language Reference	27
14.	Appendix A — Grammar Summary	29
15.	Appendix B — Examples Library	30
16.	Appendix C — Glossary	32

This document is the official reference for the LEXICO language and its compiler. It is derived from the project report, the technical architecture report, `COMPILER_GUIDE.md`, and the compiler sources in `src/`. Every feature shown here is implemented and verified against the current build.

CHAPTER 1

Introduction

1.1 What LEXICO is

LEXICO is a statically-typed, compiled, word-based programming language. Instead of symbol-heavy code, LEXICO programs read almost like English. A classical, deterministic compiler written in C++17 turns a `.lx` source into LLVM IR, verifies it, and runs it in-process with MCJIT.

```
create int price    set value 20;
create int quantity set value 5;
create int total    set value price times quantity;

if total is greater than 50 then
> print "Big order: ", total;
> print "Applying discount";
print "Done";
```

1.2 Design goals

GOAL	HOW LEXICO ACHIEVES IT
Readability	Word-based keywords (<code>plus</code> , <code>times</code> , <code>is greater than</code>)
Predictability	Deterministic pipeline; no hidden runtime magic
Teachable	A block is a line starting with <code>></code> ; nesting is more <code>></code> s
Real compilation	LLVM IR + MCJIT; not an interpreter
Forgiving errors	Optional AI assistant explains and can auto-repair
Safety first	Every heap allocation checked, bounds-checked collections

1.3 What LEXICO is *not*

LEXICO is **not** a natural language. It is a *controlled, canonical* syntax. You may write `set total to price times quantity;`, but you may not write “*please multiply price by quantity*”. The AI layer can *suggest* the canonical form, but the compiler only accepts canonical syntax.

Design principle. The compiler is deterministic and is the single source of truth. The AI is an *optional assistant* at the driver level, invoked only after a compiler failure. It never participates in parsing, type-checking, or code generation.

1.4 Feature map at a glance

T

TYPES

`int` , `float` , `char` , `string` ,
`bool`

↺

CONTROL FLOW

`if / otherwise when` , `while` ,
`for` , `repeat until` , `attempt`

f

ROUTINES

`define routine ... returns ... do` ,
`giveback` , `outcome of` , `emit`

▣

COLLECTIONS

`mesh` (typed + dynamic), `table` ,
`matrix mesh`

📄

FILES

`open ... file do` , `write` , `read` ,
`clear` , `title` , `close`

★

CLASSES

`define class ... include` ,
`inherits` , `override` , `profile`

↻

MODULES

`load "path.lx";` with cycle
detection

🔧

AI ASSIST

`--autofix` , `LEXICO_AI_HINTS=1` ,
`OPENROUTER_API_KEY`

⚙️

BACKEND

LLVM C API, IR verification, `.ll`
output, MCJIT

Installation & Setup

2.1 Requirements

TOOL	MINIMUM VERSION	PURPOSE
C++ compiler	C++17 (g++ / clang / MSVC)	Compiler core
CMake	3.20+	Build system
Flex	2.6+	Lexer generator
Bison	3.x (GLR mode)	Parser generator
LLVM	15+ (C API)	IR generation + MCJIT
nlohmann/json	header-only	AI response parsing
curl	any	Optional AI features only

2.2 Build with CMake (recommended)

```
cmake -S . -B build-cmake
cmake --build build-cmake -j
```

CMake runs Flex and Bison, discovers LLVM, links LLVM-C, and produces build-cmake/lexico (or lexico.exe on Windows). Release builds add -O2, section garbage collection and optional IPO/LTO.

2.3 Manual build on Windows (MSYS2)

```
$env:PATH = "C:\msys64\mingw64\bin;C:\msys64\usr\bin;" + $env:PATH

bison.exe -d --defines=src/lexico.tab.hpp -o src/lexico.tab.cpp src/lexico.y
flex.exe -o src/lexico.lex.cpp src/lexico.l

g++.exe -std=c++17 -Wall -Wextra -fpermissive `
  "-IC:\PROGRA~1\LLVM\include" -Isrc -D_GNU_SOURCE `
  -o build\lexico.exe `
  src\main.cpp src\driver.cpp src\ast.cpp src\syntab.cpp src\codegen.cpp `
  src\lexico.tab.cpp src\lexico.lex.cpp `
  "-LC:\PROGRA~1\LLVM\lib" -lLLVM-C
```

2.4 Running the compiler

```
# compile and run a program
lexico program.lx

# on a parse error, try an automatic repair
lexico --autofix program.lx
```

- Input files **must** have the .lx extension.
- The generated IR is written next to the build output as a .ll file.

- The program runs immediately after verification via MCJIT.

Tip. Add the folder that contains `lexico` to your `PATH` so you can invoke the compiler from any project directory.

2.5 Environment variables

VARIABLE	EFFECT
<code>OPENROUTER_API_KEY</code>	Required for any AI feature (suggestions or autofix).
<code>OPENROUTER_MODEL</code>	Optional. Default is <code>openai/gpt-4o-mini</code> .
<code>LEXICO_AI_HINTS</code>	Set to <code>1</code> to enable suggestion mode for every failure.
<code>LEXICO_SYNTAX_REF</code>	Optional path to a custom grammar reference for the AI prompt.

CHAPTER 3

Quick Start

3.1 The smallest program

```
print "hello, lexico";
```

Run it:

```
lexico hello.lx  
hello, lexico
```

3.2 Variables, arithmetic, output

```
create int price    set value 20;  
create int quantity set value 5;  
create int total    set value price times quantity;  
  
set total to total minus 5;  
  
print "total = ", total;
```

Output:

```
total = 95
```

3.3 A program with a loop

```
create int i set value 0;  
  
while i is less than 5 do  
> print "i = ", i;  
> set i to i plus 1;  
  
print "loop finished";
```

3.4 A program with a routine

```
define routine add with args in take int a, int b returns int do  
> giveback a plus b;  
  
print "1 + 2 = ", outcome of add with args in take 1, 2;
```

3.5 Style rules you must respect

Golden rules.

- Every **statement** ends with `;`.
- A **block** is opened by a header line (`if ...`, `while ...`, `define routine ...`).
- Each line **inside** a block begins with one or more `>` characters.
- The *number* of `>` characters is the nesting depth.
- Comments start with `#`.

3.6 A full, self-contained example

```
# greet.lx – read a name, greet the user, count letters
create string name set value take user input with message "Your name: ";

if length of name is greater than 0 then
> print "Hello, ", name, "!";
> print "Your name has ", length of name, " characters.";
otherwise
> print "No name given.";
```

Language Fundamentals

4.1 Comments

```
# This is a comment
create int x set value 1;  # inline comment
```

4.2 Types

LEXICO TYPE	COMPILED AS	DEFAULT VALUE	LITERAL EXAMPLE
int	i64	0	42
float	double	0.0	12.35
char	i8	'\0'	'x'
string	i8*	""	"text"
bool	i1	false	true / false

4.3 Declaring variables

```
create int a set value 20;           # one variable
create int x, y, z set value 1, 2, 3; # parallel lists
create string name set value "lexico";
create int empty;                   # default 0
create bool ready;                  # default false
```

- `create <type> <id>;` — one variable with default value.
- `create <type> <id> set value <expr>;` — one variable with initializer.
- `create <type> <id> , <id> ... set value <expr> , <expr> ...` — parallel lists; counts must match.

4.4 Assigning values

```
set a to 10;
set a to a plus 1;
set total to price times quantity;
set avg to (a plus b) divided by 2;
```

4.5 The compound self-assignment shortcut

When the target variable is the same as the first operand, you may drop `set ... to` entirely:

```
a plus 3;           # means: set a to a plus 3;
a minus 1;          # means: set a to a minus 1;
a times 2;          # means: set a to a times 2;
a divided by 4;     # means: set a to a divided by 4;
a mod 5;            # means: set a to a mod 5;
```

The right-hand side is a full expression and respects normal precedence:


```
a plus 3 times (4 divided by 2); # a = a + (3 * (4 / 2)) = a + 6
a plus 3 times 4;                # a = a + (3 * 4)      = a + 12
```

Both forms compile to identical LLVM IR.

4.6 Type conversion

```
create int n set value 65;
convert n to char;      # n is now 'A'
print n;                # A

convert n to string;    # n is now the string "A"
print n;                # A
```

- The target must be a **scalar variable**.
- Destination types: `int`, `float`, `char`, `string`.
- The variable's declared type **changes** from that point on.
- `string → int` / `float` / `char` is validated at runtime.

4.7 Reading user input

```
create int age set value take user input;

create int limit set value 10;
create int age2 set value take user input with message "Age must be > ", limit, " : ";

set age to take user input with message "Retry age: ";
```

The `with message` list is printed left-to-right before the read. Each part is any expression. A trailing comma is invalid.

Statements in Depth

5.1 Blocks and > markers

LEXICO has no braces and no `end` keyword. A block is a set of lines that start with one or more `>` markers. The count of `>` is the nesting depth.

```
if A is greater than 20 then
> print "L1";
> if A is greater than 30 then
>> print "L2";
> print "back to L1";
print "outside";
```

- 1. Only the **start of a line** matters for depth.
- 2. A block **continues** while the depth is $\geq$ the block's depth.
- 3. A block **ends** when a non-empty line has **lower depth** (or at EOF).
- 4. Blank lines do not affect depth.
- 5. Every statement still ends with `;`.

Not a shell prompt. The `>` character used for blocks is a LEXICO marker, not a terminal prompt. It must be written inside the `.lx` file.

5.2 if / otherwise

```
if total is greater than 100 then
> print "Premium";
otherwise when total is greater than 50 then
> print "Standard";
otherwise
> print "Basic";
```

ENGLISH PHRASE	MEANING
is equal to	==
is not equal to	!=
is greater than	>
is less than	<
is greater or equal to	>=
is less or equal to	<=

Boolean connectives: `and`, `or`, `not`. Parentheses are allowed.

```
if (a is greater than 0) and not (a is equal to 10) then
> print a;
```

5.3 **while** — pre-condition loop

```
create int i set value 0;

while i is less than 5 do
> print i;
> set i to i plus 1;
```

The body may run **zero** times.

5.4 **for** — counting loop

```
for i from 0 to 10 step plus 2 do
> print i;

for k from 5 to 1 step minus 1 do
> print k;
```

- **start**, **end**, **step** are evaluated **once** before the loop.
- **step** is written with **plus** / **minus** prefixes.
- **step** is optional; default is **step plus 1**.
- **step == 0** is a semantic error.
- The loop iterator is **loop-local**.

Collection-bound form:

```
for i in nums from 0 to size of nums minus 1 do
> print nums at i;
```

5.5 **repeat until** — post-condition loop

```
create int x set value 0;

repeat
> print x;
> set x to x plus 1;
until x is greater or equal to 3 then
```

The body always runs **at least once**.

5.6 **attempt** — bounded retry loop

```
attempt up to 3 while guess is not equal to secret do
> print "Trying...";
> set guess to guess plus 3;
on failure
> print "I gave up after 3 tries.";
```

- Run the body while the condition is true, but **at most N times**.
- **on failure** runs **once** if the loop exhausted all attempts and the condition is still true.
- If the loop exits early (condition became false), **on failure** is **skipped**.

5.7 `print`

```
print "name = ", name, " age = ", age;  
print a plus 3, " is the sum";  
print nums;           # prints a table-style view of a mesh
```

`print` accepts a comma-separated list of expressions; each is evaluated left-to-right. A newline is printed at the end.

Expressions and Operators

6.1 Arithmetic

WORD	SYMBOL	NOTES
plus	+	
minus	-	binary and unary
times	*	
divided by	/	integer division on int
mod	%	

Precedence (highest → lowest):

1. Parentheses ()
2. times, divided by, mod
3. plus, minus

```
print 2 plus 3 times 4;           # 14
print (2 plus 3) times 4;        # 20
print 10 divided by 3;           # 3
print 10 mod 3;                  # 1
```

6.2 Strings

```
set s to "hello" plus "world";   # "helloworld"
set s to "banana" minus "na";    # "bana"
set s to "abc" minus "z";        # "abc" (no change)
```

Only plus and minus are allowed on strings.

6.3 Strings as indexed sequences

```
create string title set value "lexico";
print title at 2;                # 'x'
print length of title;           # 6
```

6.4 Extraction and membership

```
extract "hello" from source;     # "hello" if present, else ""
extract from 2 to 5 in source;    # substring slice
check "abc" in source;            # bool: uses strstr
position of "abc" in source;      # first index or -1
```

6.5 Conversions in expressions

```
print convert 65 to char;      # 'A'  
print convert "42" to int;    # 42  
print convert 3.75 to int;    # 3
```

6.6 Expression modifiers on collections

```
print size of nums;           # element count  
print count 3 in nums;        # number of 3s  
print check 3 in nums;        # true / false  
print position of 3 in nums;   # first index or -1  
print row length of m;        # matrix rows  
print column length of m;     # matrix cols
```

CHAPTER 7

Collections: Meshes, Tables, Matrices

7.1 `mesh` — the canonical collection

LEXICO unifies all collections under `mesh`. There are two forms.

Typed mesh (homogeneous element type)

```
create int    mesh nums    sized 10;
create string mesh names   sized 5;
create float  mesh prices  sized base plus 5;
```

Dynamic mesh (mixed scalar types)

```
create mesh mixed sized 12;
```

Auto-sized by values

```
create int    mesh nums      set values 12, 32, 12, 54, 25;
create string mesh words     set values "alpha", "beta", "gamma";
create mesh mixedAuto set values 12, "hi", 'x', 3.14, true;
```

7.2 Reading and writing

```
print nums at 2;           # read by index
set nums value at 2 to 99;  # write by index
set nums value at 3 to take user input; # read from stdin
```

7.3 Appending

```
append 100 to nums;           # append at end
append 50 to nums at 0;       # insert at index 0 (shifts right)
```

Inside a `for i in nums ...` loop, use the contextual form:

```
for i in nums from 0 to size of nums minus 1 do
> set value to 42;           # writes nums[i]
> append 7;                  # appends to nums
```

7.4 Sorting

```
sort nums    ascendantly;
sort words   descendantly;
```

Sorting works only on typed meshes with element type `int`, `float`, `char`, or `string`. Dynamic meshes are rejected.

7.5 Mesh queries

EXPRESSION	RESULT
<code>size of nums</code>	element count (<code>int</code>)
<code>count v in nums</code>	number of elements equal to <code>v</code>
<code>check v in nums</code>	<code>true</code> if at least one match
<code>position of v in nums</code>	first index or <code>-1</code>

7.6 Matrices (`matrix mesh`)

Matrices are 2-D typed or dynamic collections.

Declaration

```
create int    matrix mesh m      rows 2 cols 3;
create string matrix mesh labels rows 4 cols 2;
create       matrix mesh mixed  rows 2 cols 2;
create int    matrix mesh a set values 1,2,3 / 4,5,6;
```

Indexed access

```
set m at 1,2 to 6;
set x to m at 2,1;
```

Printing

```
print m;
```

Output shape:

```
m[2x3]
1      2      3
4      5      6
```

Matrix loop

```
for i j in m i from 0 to 1 and j from 0 to 2 do
> set i plus j at i,j;      # write
> set x at i,j;            # read into scalar x
```

Size expressions

```
print size of m;           # occupied slots
print row length of m;     # effective row count
print column length of m;  # column count
```


CHAPTER 8

Routines and Classes

8.1 Defining a routine

```
define routine add with args in take int a, int b returns int do
> giveback a plus b;
```

- `define routine` — global routine.
- `with args in take ...` — parameter list (optional).
- `returns <type>` — return type (mandatory).
- `do` — body follows as a `>` block.
- `giveback <expr>;` — returns a value.

8.2 Calling a routine

```
emit add with args in take 1, 2; # discard result
create int z set value outcome of add with args in take 5, 7;
print outcome of add with args in take z, 3;
```

- `emit` — statement; result is discarded.
- `outcome of` — expression; result is a value.

8.3 Classes

```
define class Shape include
> create string label;
> profile base do
>> set label to "generic-shape";
> routine show do
>> print "Shape: ", label, "\n";

define class Circle inherits Shape include
> create int radius;
> profile default do
>> set label to "circle";
>> set radius to 5;
> override routine show do
>> print "Circle of radius ", radius, "\n";

create Circle myCircle using default;
emit myCircle_show;
```

- `define class <Name> include` — class body.
- `inherits <Parent>` — inherits fields and routines.
- `override create <type> <field>;` — replace inherited field.
- `override routine <name> ... do` — replace inherited routine.
- `profile <name> do` — constructor profile; instantiate with `create <Class> <var> using <profile>;`.
- Profile names must be non-keyword identifiers.

8.4 Method calls

After `create Circle c using default;`, methods are called with `<var>_<method>`:

```
emit c_show;  
create int area set value outcome of c_describe;
```

CHAPTER 9

File I/O

9.1 Opening a file

```
open "notes.txt" file do
> ...           # file is open here

open "notes.txt" file make do
> ...           # create if it doesn't exist
```

The file is closed automatically at the end of the block.

9.2 Writing

```
write "alpha" in file;           # append line at end
write "alpha" in line 2 in file;  # replace line 2 (0-based)
```

9.3 Reading

```
read file into rows;           # table of all lines
read line 3 in file into one_line; # one string
read 5 lines in file into rows;  # first 5 lines
read 4 lines from line 2 in file into rows; # range window
```

- Single-line reads require a `string` scalar target.
- Multi-line reads require a `table` target.

9.4 Clearing

```
clear file;           # truncate the whole file
clear line 3 in file;  # remove one line
clear from line 1 to line 4 in file; # inclusive range
```

9.5 Renaming and closing

```
set file title to "notes_renamed.txt";
close file;           # safe; auto-close is idempotent
```

9.6 Full example

```
create string one_line set value "";
create table rows with size 0;

open "notes.txt" file make do
> clear file;
> write "alpha" in file;
> write "beta" in file;
> write "BETA-UPDATED" in line 1 in file;
>
> read line 1 in file into one_line;
> print one_line;
>
> read file into rows;
> print rows;
>
> clear line 0 in file;
> read file into rows;
> print rows;
>
> set file title to "notes_renamed.txt";

open "notes_renamed.txt" file do
> read file into rows;
> print rows;
```

CHAPTER 10

Modules — load

Split a project across files with `load` :

```
load "lib/classes.lx";
load "lib/routines.lx";

create Dog d using default;
emit d_bark;
print outcome of add with args in take 4, 6;
```

- Loaded files **must** have the `.lx` extension.
- Relative paths are resolved from the directory of the **loading file**.
- Absolute paths are accepted.
- Duplicate loads are silently ignored.
- Circular loads are detected and reported as a compiler error.
- Symbols merge into a single compilation unit.

10.1 Best practices

- Put `load` statements at top-level, usually near the beginning of the file.
- Keep reusable classes/routines in library-style files.
- Keep entry logic in a separate main file.

Example layout.

```
project/
├─ main.lx
└─ lib/
   ├─ classes.lx
   └─ routines.lx
```

CHAPTER 11

AI-Assisted Error Recovery

11.1 What it is

When a compiler stage fails, LEXICO can call a Large Language Model (via OpenRouter) to **explain the error** and, for parse errors, to **propose a corrected source file**. The AI layer is entirely optional. The deterministic compiler remains the source of truth.

MODE	TRIGGER	BEHAVIOUR
Suggestion	Any parse / semantic / codegen failure	Prints an explanation and a fix hint
Autofix	Parse error, <code>--autofix</code> given	Tries a local fix; if that fails, asks the AI for a corrected file; validates, writes, retries once

11.2 Enabling it

```
# required for any AI feature
export OPENROUTER_API_KEY="your-key"

# optional - choose the model
export OPENROUTER_MODEL="provider/model"

# enable suggestion mode (never writes to disk)
export LEXICO_AI_HINTS=1
```

Then run:

```
lexico --autofix program.lx
```

Never commit your API key. Keep it in your shell environment or a secrets manager, never inside `.lx` files or your repository.

11.3 How autofix works — step by step



11.4 Before / After — annotated examples

Example 1 — Missing semicolon

Source (`demo1.lx`) — before:

BEFORE — DEMO1.LX

```
create int a set value 20
print a;
```

AFTER — DEMO1.LX

```
create int a set value 20;
print a;
```

Compiler output — before autofix:

```

lexico demo1.lx

Compiler: parsing source (demo1.lx).
Parse error (line 2): syntax error, unexpected PRINT, expecting ';' or ...
Compilation aborted due to parse errors.
Fix hint: Missing ';' at end of line 1.
```

Compiler output — with `--autofix`:

```
lexico --autofix demo1.lx

Compiler: parsing source (demo1.lx).
Parse error (line 2): syntax error, unexpected PRINT, expecting ';' or ...
Fix hint: Missing ';' at end of line 1.
Autofix explanation: Parser expected ';' before the next statement. This fix inserts the
missing semicolon.
Fix preview around line 2:
  before: create int a set value 20
  after : create int a set value 20;
Apply this fix to 'demo1.lx'? [y/N]: y
Autofix applied to 'demo1.lx'. Retrying compile...
Compiler: parse success.
Compiler: semantic analysis success.
Compiler: code generation success (demo1.ll).
20
```

Example 2 — Missing `by` after `divided`

BEFORE	AFTER
<pre>create int a set value 10; create int b set value 2; create int c set value a divided 2; print c;</pre>	<pre>create int a set value 10; create int b set value 2; create int c set value a divided by 2; print c;</pre>

```
lexico --autofix demo2.lx

Compiler: parsing source (demo2.lx).
Parse error (line 3): syntax error, unexpected INT_LIT, expecting ...
Fix hint: Missing 'by' after 'divided' on line 3.
Autofix applied to 'demo2.lx'. Retrying compile...
Compiler: parse success.
Compiler: semantic analysis success.
Compiler: code generation success (demo2.ll).
5
```

Example 3 — AI-assisted fix for a deeper syntax error

The author mistakenly wrote `+` (a symbol) instead of the keyword `plus`.

BEFORE	AFTER
<pre>define routine add with args in take int a, int b > giveback a + b; print outcome of add with args in take 1, 2;</pre>	<pre>define routine add with args in take int a, int b > giveback a plus b; print outcome of add with args in take 1, 2;</pre>


```
lexico --autofix demo3.lx

Compiler: parsing source (demo3.lx).
Parse error (line 2): syntax error, unexpected '+', expecting ...
Fix hint: Check syntax near line 2.
Autofix explanation: AI proposed a parse-error fix based on your file and grammar context.
Fix preview around line 2:
  before: > giveback a + b;
  after  : > giveback a plus b;
Apply this fix to 'demo3.lx'? [y/N]: y
Autofix applied to 'demo3.lx'. Retrying compile...
Compiler: parse success.
Compiler: semantic analysis success.
Compiler: code generation success (demo3.ll).
3
```

11.5 Safeguards

- **API key check.** No key → no network call.
- **Deterministic-first.** Local heuristics run before any API call.
- **Response validation.** `looks_like_lexico_source()` rejects JSON payloads, error messages, and obviously invalid text.
- **Whitespace-only guard.** A diff that only changes whitespace is discarded.
- **Single retry.** The compiler will not loop.
- **Interactive confirmation.** On a TTY, the user must type `y`.
- **Grammar grounding.** The prompt includes parser states from `src/lexico.output`.

11.6 Limits of the AI layer

- Requires network and a valid key.
- Suggestion quality varies by model.
- **Weakest** on context-sensitive semantic errors.
- Not guaranteed minimal or always correct — the deterministic compiler still validates the fix.

Example — semantic error (suggestion mode)

SOURCE	AI SUGGESTION
<pre>create int a set value 1; create string s set value "hi"; set a to s;</pre>	<pre>Root cause: assigning a string to an int variable. Fix: change the target's type or convert first. create int a set value 1; create string s set value "hi"; set s to convert a to string;</pre>

Diagnostics Reference

LEXICO reports errors with a **stage**, a **file**, a **line**, and a message.

STAGE	EXAMPLE MESSAGE	TYPICAL CAUSE
Lexical	Error (line N, col M): unexpected character '#'	Invalid symbol
Lexical	Error (line N, col M): token too long	Oversized identifier
Parse	Parse error (line N): syntax error, unexpected X, expecting Y	Missing <code>;</code> , wrong keyword
Semantic	Error (line N): use of undeclared identifier 'x'	Name not declared
Semantic	Error (line N): assignment to 'a' has type string, expected int	Type mismatch
Semantic	Error (line N): write value must be string	Non-string write
Runtime	Runtime error: index 5 out of bounds [0..3)	Bad index
Runtime	Runtime error: cannot convert 'abc' to int	Bad numeric conversion
Runtime	Runtime error: cannot open file 'notes.txt'	File I/O failure

12.1 Diagnostic order

Diagnostics are always reported in this order:

1. Lexer
2. Parser
3. Semantic
4. Runtime

12.2 Anatomy of a diagnostic

anatomy of a diagnostic

Error (line 3): assignment to 'a' has type string, expected int.

↑

↑

↑

└─ why it failed

└─ which identifier

└─ source line number

Complete Language Reference

13.1 Keywords (reserved)

create set value values print convert
append sort ascendantly descendantly count
check position extract define class
include inherits profile using routine
override returns emit giveback outcome
args in take user input with message to
plus minus times mod divided by if then
otherwise when and or not while do for
from step repeat until attempt up to on
failure times while open make file write
read line lines clear begin close title
into array table mesh matrix rows cols
row column of size length sized at each
index load int float char string bool
true false is equal to is not equal to is
greater than is less than is greater or equal
to is less or equal to

13.2 Statement summary

STATEMENT	FORM
Declaration	<code>create <type> <id> [, <id> ...] [set value <expr> [, <expr> ...]];</code>
Assignment	<code>set <id> to <expr>;</code>
Compound shortcut	<code><id> <op> <expr>;</code>
Conversion	<code>convert <id> to <type>;</code>
Input	<code>set <id> to take user input [with message <parts>;]</code>
Print	<code>print <expr> [, <expr> ...];</code>
If	<code>if <cond> then <block> [otherwise ...]</code>
While	<code>while <cond> do <block></code>
For	<code>for <i> from <a> to [step <expr>] do <block></code>
For-each	<code>for <i> in <mesh> from <a> to [step <expr>] do <block></code>
Repeat	<code>repeat <block> until <cond> then</code>
Attempt	<code>attempt up to <n> while <cond> do <block> [on failure <block>]</code>
Routine	<code>define routine <name> [with args in take <p> ...] returns <t> do <block></code>
Emit	<code>emit <name> [with args in take <expr> ...];</code>
Giveback	<code>giveback <expr>;</code>
File open	<code>open <path> file [make] do <block></code>
File write	<code>write <expr> in file [in line <n>;]</code>
File read	<code>read file into <id>;</code> (and variants)
File clear	<code>clear file;</code> / <code>clear line <n>;</code> / <code>clear from line a to line b;</code>
File title	<code>set file title to <expr>;</code>
File close	<code>close file;</code>
Class	<code>define class <name> [inherits <parent>] include <body></code>
Load	<code>load "<path>.lx";</code>

13.3 Literals

KIND	EXAMPLES
Integer	<code>42</code> , <code>-7</code>
Float	<code>3.14</code> , <code>-0.5</code>
Char	<code>'a'</code> , <code>'\n'</code> , <code>'\t'</code> , <code>'\"'</code> , <code>'\\'</code> , <code>'\0'</code>
String	<code>"hello"</code> , <code>""</code>
Bool	<code>true</code> , <code>false</code>

APPENDIX A

Grammar Summary

The full canonical grammar is maintained in [src/lexico.y](#). A high-level sketch is presented below.

```
program      : /* empty */ | stmt_list ;
stmt_list    : stmt | stmt_list stmt ;
stmt         : decl_stmt ';' | print_stmt ';' | set_stmt ';'
             | convert_stmt ';' | emit_stmt ';' | giveback_stmt ';'
             | compound_stmt ';' | if_stmt | while_stmt | for_stmt
             | repeat_stmt | attempt_stmt | function_stmt
             | class_stmt | file_open_stmt | file_stmt ';'
             | collection_decl_stmt ';' | collection_set_stmt ';'
             | append_stmt ';' | sort_stmt ';' ;

decl_stmt    : CREATE type IDENT [SET VALUE expr] ;
set_stmt     : SET IDENT TO expr | SET IDENT AT expr ',' expr ;
convert_stmt : CONVERT IDENT TO type ;
while_stmt   : WHILE condition DO block ;
for_stmt     : FOR IDENT FROM expr TO expr [STEP expr] DO block
             | FOR IDENT IN IDENT FROM expr TO expr [STEP expr] DO block ;
if_stmt      : IF condition THEN block [else_part] ;
block        : INDENT block_body DEDENT ;
condition    : cond_or ;
cond_or      : cond_or OR cond_and | cond_and ;
cond_and     : cond_and AND cond_not | cond_not ;
cond_not     : NOT cond_atom | cond_atom ;
cond_atom    : expr CMP_xx expr | '(' condition ')' ;
```

Details of every production appear in [src/lexico.y](#).

APPENDIX B

Examples Library

B.1 Hello, world

```
print "hello, world";
```

B.2 FizzBuzz

```
for i from 1 to 15 do
> if i mod 15 is equal to 0 then
>> print "FizzBuzz";
> otherwise when i mod 3 is equal to 0 then
>> print "Fizz";
> otherwise when i mod 5 is equal to 0 then
>> print "Buzz";
> otherwise
>> print i;
```

B.3 Palindrome check

```
create string s set value take user input with message "Word: ";
create string rev set value "";

for i from length of s minus 1 to 0 step minus 1 do
> set rev to rev plus s at i;

if rev is equal to s then
> print "Palindrome";
otherwise
> print "Not a palindrome";
```

B.4 Retry loop with user input

```
create int secret set value 7;
create int guess set value 0;
create int tries set value 0;

attempt up to 3 while guess is not equal to secret do
> set guess to take user input with message "Guess: ";
> set tries to tries plus 1;

on failure
> print "Failed after ", tries, " tries.";
```

B.5 Mesh statistics

```
create int mesh nums set values 4, 8, 15, 16, 23, 42;
create int total set value 0;

for i in nums from 0 to size of nums minus 1 do
> set total to total plus nums at i;

sort nums ascendantly;

print "sum = ", total;
print "min = ", nums at 0;
print "max = ", nums at size of nums minus 1;
```

B.6 Matrix initialisation

```
create int matrix mesh m set values 1,2,3 / 4,5,6;
print m;

for i j in m i from 0 to 1 and j from 0 to 2 do
> set i times 10 plus j at i,j;

print m;
```

B.7 Password manager (skeleton)

```
create string user set value take user input with message "User: ";
create string pass set value take user input with message "Pass: ";

open "vault.txt" file make do
> write user in file;
> write pass in file;

open "vault.txt" file do
> create table lines with size 0;
> read file into lines;
> print lines;
```

Glossary & Credits

Glossary

TERM	MEANING
AST	Abstract Syntax Tree — the tree produced by the parser.
Driver	The orchestration layer that runs the pipeline.
GLR parser	A generalised LR parser (Bison). Used to allow natural comparisons.
INDENT / DEDENT	Virtual tokens emitted by the lexer for <code>></code> blocks.
MCJIT	LLVM's in-process just-in-time execution engine.
Mesh	The canonical LEXICO collection (typed or dynamic).
Matrix mesh	A 2-D collection with row and column indices.
Profile	A class constructor-like block that initialises fields.
Routine	A named, typed, callable block of logic.
Symbol table	The structure holding variable, collection, and function information.
Autofix	The optional AI-driven repair loop for parse errors.
Grammar grounding	Feeding the LLM parser states so its fixes match the real grammar.

Credits

Author	Haythem Gharbi
Supervisor	Prof. Jallouli Malika
Institution	Faculty of Sciences of Monastir — Software Engineering
Academic year	2025 – 2026
GitHub	github.com/haythemgharbio1
Backend	LLVM C API, MCJIT
Frontend	Flex + Bison (GLR)
AI provider	OpenRouter (optional)

Sources. This documentation is assembled from `README.md`, `COMPILER_GUIDE.md`, `AI_COMPILER_ARCHITECTURE_REPORT.md`, and the compiler sources in `src/` (`lexico.l`, `lexico.y`, `ast.*`, `syntab.*`, `codegen.*`, `driver.*`).